Resource-Efficient Deployment of Deep Learning Models for Semiconductor Defect Classification on Embedded Devices

Wei-Xuan Lim*†, Marco Wagner†, Cheng Yee Low* and Uwe Gleiter†
*Universiti Tun Hussein Onn Malaysia, Email: hd230012@student.uthm.edu.my, cylow@uthm.edu.my
†Heilbronn University of Applied Sciences, Germany, Email: {marco.wagner, uwe.gleiter}@hs-heilbronn.de

2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON) | 979-8-3315-2684-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/MELECON64486.2026.11418805

***Abstract*—Powerful deep learning (DL) models for semiconductor defect classification are computationally infeasible for deployment on low-power edge devices. This research addresses this gap by analyzing the effects of pruning and post-training quantization on model accuracy, latency, and power consumption on a Raspberry Pi. Through multi-objective analysis, we found that 8-bit integer quantization proved highly effective, reducing inference latency by 41% (from 3.06s to 1.79s) and average power consumption by 50% (from 15.9W to 7.6W), while maintaining or slightly improving accuracy to 94.1%. The optimal deployment configuration, which combines 8-bit quantization with moderate pruning (40%-60% sparsity), reduces energy per inference by approx. 3.5 times and decreases model size from 90MB to 25MB. These findings validate that production-grade accuracy can be maintained, enabling practical, on-device defect detection under stringent edge computing budgets.**

***Index Terms*—deep learning, semiconductor, surface defects, ResNet, pruning, optimization, quantization, Raspberry Pi**

I. Introduction

In the semiconductor industry, product reliability hinges on the early detection of surface anomalies. Even microscopic defects—such as scratches, cracks, or foreign materials—can render end-user applications non-functional. Consequently, Automated Optical Inspection (AOI) has become the industry standard, replacing manual verification to ensure high throughput and minimize human error. While traditional AOI provides a baseline for quality control, the recent integration of Deep Learning (DL) has revolutionized the field, offering superior precision in defect classification and significantly reducing false negative rates.However, the adoption of these advanced DL models introduces a critical operational bottleneck, computational intensity. State-of-the-art algorithms typically require substantial processing power, making them ill-suited for the resource-constrained edge devices often found on industrial factory floors. A significant gap currently exists in the literature between the theoretical development of high-accuracy models and their practical deployment in power-sensitive, low-latency environments.

This paper addresses that disparity by evaluating the trade-off between algorithmic complexity and industrial practicability. We present a systematic comparative study of three model optimization techniques—pruning, post-training quantization, and knowledge distillation—applied specifically to the domain of binary chip defect classification. By deploying these optimized models on a low-power Raspberry Pi, we demonstrate a viable methodology for maintaining high inference accuracy while minimizing computational overhead. This work contributes a validated framework for bringing deep learning out of the server room and onto the manufacturing edge.

The remainder of this paper is structured as follows: Section II reviews related work concerning DL for defect classification and model optimization on embedded devices. Section III details the experimental setup, including model training, hardware configuration (Raspberry Pi), power consumption measurement, and the application of compression techniques. Section IV presents the results and analysis of the effects of each optimization on model accuracy, inference time, and power consumption. Finally, Section V concludes the paper by summarizing the key findings and suggesting avenues for future research.

II. Literature Review

A. Advances in Deep Learning Models for Surface Defect Classification

The semiconductor industry has significantly advanced its AOI through the adoption of deep learning models for surface defect classification, thereby replacing subjective and inefficient manual inspection methods [1]. Early implementations leveraged Convolutional Neural Networks (CNNs), often via transfer learning, to achieve near human-level performance, demonstrating a high classification accuracy of 87.26% and contributing to reduced labor costs [2]. However, a key challenge in this domain is the model's inability to accurately classify unseen defects encountered post-training, which often leads to decreased performance. To address this, approaches integrating methods like k-nearest neighbors (k-NN) with CNNs have been developed, enabling the classification of unknown defects without extensive retraining and achieving improved overall accuracy, up to 96.2% [3]. The inherent complexity of semiconductor products, including diverse defect types and intricate background inference, has necessitated the development of deeper and larger DL models [4]. While techniques such as adding dropout layers [5] or custom layers (Average Pooling, Dense) [6] have been introduced to mitigate overfitting and push prediction accuracy to 97.00% and even 100%, respectively, these modifications inherently increase the model size. Further complexity has been introduced through

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.

979-8-3315-2684-9/26/$31.00 ©2026 IEEE

custom architectures, such as VGG16 with added convolutional layers and concatenation, achieving 97.88% accuracy [7]. More recently, hybrid methods combining various DL models and algorithms have been proposed to boost classification performance for the most challenging defects. Examples include integrating YOLOv3 with a residual network for multi-level feature learning, resulting in 99.2% accuracy [8], and complex multi-stage systems, such as a hybrid CNN combining VGG with Stacked What-Where Autoencoders (SWWAE) and residual learning, yielding an F1-score of 99.5% [9]. Another hybrid approach combining a Faster R-CNN detector with a ResNet-50 classifier achieved 93.69% accuracy [10]. While these sophisticated and complex architectures have established state-of-the-art accuracy, their size and computational demands have significantly increased, leading to a critical need for substantial computational resources.

B. Cascaded Framework of Binary Classification and Multi Class Defect Detection

Residual Networks (ResNets) have become the dominant architectural choice for semiconductor surface defect classification due to their powerful feature extraction, strong generalization, and high accuracy across diverse inputs, including Scanning Electron Microscope (SEM) images and chip surface data. Variants such as ResNet-34, ResNet-50, and ResNet-152 have achieved industrial-level performance, often enhanced by techniques like attention mechanisms and transfer learning [1]–[3]. Table I summarizes the reported performance of various ResNet architectures in the recent literature. While these high accuracies are crucial for industrial standards, these gains are inherently tied to an increase in model parameters and network depth, leading to significant computational overhead.

TABLE I
THE PERFORMANCE COMPARISON OF DIFFERENT RESNET VARIANTS

Ref	Model	Database	Acc. (%)
López de la Rosa et al., 2022 [11]	ResNet-50	Real, private SEM images	98.0
Nag et al., 2022 [12]	ResNet-50	Mixed WM-38K	98.2
Wang et al., 2022 [13]	ResNet-50	Real, private Optical Microscopy images	93.6
X. Zheng et al., 2023 [14]	ResNet-34	Real, private chip images	87.3
Zhang et al., 2024 [15]	ResNet-34	Real, private SMT images	98.0
Na et al., 2024 [16]	ResNet-50	Real, private SEM images	98.0
Kim et al., 2023 [17]	ResNet-50	Real, private SEM images	77.4
Jeong et al., 2024 [6]	ResNet-50	Real, private Wafer Edge images	100.0

To mitigate the computational demands, research has moved towards more efficient frameworks. For instance, Zhu et al. proposed a cascaded framework that initially uses a ResNet-50 model for binary classification (good vs. defective), followed by the use of YoloX with a shallow-to-deep attentional feature fusion (SDAFF) mechanism for subsequent defect detection and localization [18]. This approach aims to structure the task for better resource management.

C. Limitations of Large-Scale Models for Embedded Devices

Despite advancements like the cascaded framework, the fundamental challenge of high computational cost persists due to the reliance on large backbone models like ResNet-50 for initial classification [18]. The sheer size of these models significantly increases latency, power consumption, and makes deployment challenging on resource-constrained edge devices such as the Nvidia Jetson Nano, and Raspberry Pi [19]. Even with the advent of embedded devices featuring hardware accelerators and multi-core processors, their limited computational power, memory, and memory bandwidth are often insufficient to support recent deep learning model variants. For context, the ResNet-50 model contains 25.6 million parameters, requiring approximately 87MB of memory and 3.8 billion floating-point operations (FLOPs) for a single image classification. A comparison of the typical computational and memory demands of various deep learning models is summarized in Table II. This highlights the necessity for model optimization to enable practical real-time inference on low-power industrial hardware.

TABLE II
COMPARISON OF TYPICAL DEEP LEARNING MODELS [19]

Model	Model Performance Metrics		
	Accuracy (Top-1/Top-5, %)	*Parameters (M / MB)*	*FLOPs (G)*
AlexNet	57.0 / 80.3	61 / 238	0.72
VGG-16	70.5 / 90.0	138 / 528	15.5
GoogLeNet	72.5 / 90.8	6.9 / 90	1.6
ResNet-18	70.7 / 89.9	11.2 / 44	1.8
ResNet-50	75.8 / 92.9	25.6 / 87	3.8
MobileNet	70.4 / 89.5	4.2 / 16	0.57

D. Model Compression using Quantization and Pruning Techniques

The heavy computational footprint of high-accuracy deep learning models, evidenced by their large parameter counts (Table II), necessitates aggressive model optimization for deployment on resource-constrained edge devices [19], [20]. Key optimization strategies include pruning, quantization, and knowledge distillation. Pruning is a vital compression method that removes redundant and less important parameters from a pre-trained network. Early work by LeCun et al. introduced unstructured pruning—the removal of individual weights below a certain magnitude—to reduce model size without significant accuracy loss [21]. However, this often results in irregular sparsity, hindering hardware acceleration due to memory and cache issues. Consequently, research diversified into two main categories: Unstructured Pruning techniques focused on refining weight removal (e.g. [22], [23], [24]) and Structured Pruning which removes entire structural units (e.g. kernel-level pruning [25], filter-level pruning [26],

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.

next-layer statistics (ThiNet) [27], AutoPruner [28]).Quantization is a technique that represents the weights and activations of models using fewer bits, thereby reducing model parameters and overall complexity. Suyog et al proved that the weights of deep networks that represented by 16-bits floating points will not affect the classification accuracy [29]. Courbariaux et al showed that dynamic fixed point representation is superior than floating point and fixed point [20]. Mathew et al. introduced a quick sparsity and quantization technique through 8-bit representation by modifying the Caffe CNN model [30]. Kluska and Zieba perform a post-training quantization method for convolutional neural network for detection and classification task which able to compress neural networks up to 87% while maintaining desired level of accuracy [31].

III. Methodolgy

This section details the experimental setup, the software configuration used, the specifications of the datasets, the adopted base deep learning architecture, and the specific application of pruning and quantization techniques to optimize the model.

A. Hardware Setup

For this experimental setup, we utilized the Raspberry Pi 4 Model B due to its rich set of peripheral ports and GPIO pins, which facilitates easy configuration and direct connection to external sensors or actuators. The device runs the official Raspberry Pi OS (based on Debian Linux) and is powered via a 5V supply. Given the device's limited computational resources, it is configured for offline inference of the binary classification task, supporting standard deep learning APIs such as Python and TensorFlow Lite, with XNNPACK providing optimized kernel execution for efficient inference on embedded devices.

The APIs and modules installed are highly dependent on the architecture of the deep learning models and the operating system of the Raspberry Pi. Hence, before installing the APIs, a virtual environment had been created for installing the external packages and libraries to carry out the testing and evaluation. This will help to prevent the dependency issues from the pre-installed packages.

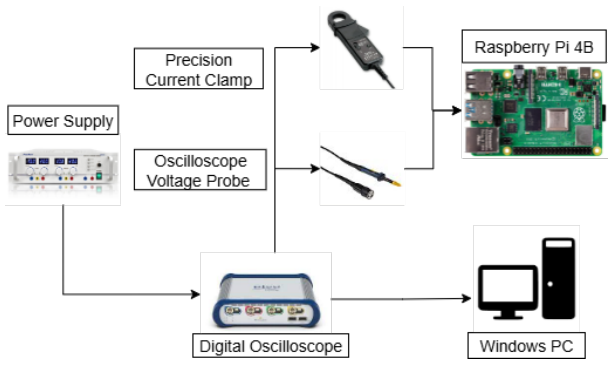


Fig. 1. Experimental setup used for power measurement.

The power measurement setup (Figure 1) was designed to evaluate the deep learning model's power consumption and latency on the Raspberry Pi 4B. A stable 5V DC power supply was used to power the device. To measure dynamic current draw, a Pico TA series precision current clamp was connected in series with the power supply line. A GPIO pin on the Raspberry Pi was set from LOW to HIGH immediately before each test image inference and back to LOW afterwards, creating a discernible current spike. The current waveform was digitized in real-time by a PicoScope 6000 Series and displayed on a PC. This setup allows for both observing the sharp increase in current during the image classification task compared to the idle baseline current as well as collecting data to calculate power consumption, energy per inference, and overall efficiency.

The complete, labeled dataset utilized in this study is sourced from Zhu et al. [18]. The raw data contained 2270 nondefective and 1241 defective samples. To address the inherent class imbalance typical of real-world defect data, we employ a joint, balanced dataset. This dataset was created using semantic-based generation and synthetic augmentation to achieve class parity. It includes instances of four specific defect categories (pollution, crack, broken, and no die), all consolidated into the defective class for the primary objective of binary classification against the non-defective class.

B. Optimization Techniques on the ResNet-50 Model

In our study, we base on the cascaded model approach by Zhu et al. using a ResNet-50 model as a first stage to classify between defective and non-defective samples and a second stage which is only used for defective samples and leverages a YoloX-SDAFF model to determine the actual defect [18]. Since this second stage is called rather infrequently in real-world scenarios, our work focuses on the first stage and tries to optimize the ResNet-50 model. After finalizing the training of the ResNet-50 model, the following model compression techniques are used:

- TensorFlow pruning in Keras to conduct a magnitude-based weight training setting the smallest absolute weights to zero. The pruning will be done gradually through several epochs via pruning schedule with polynomial decay. Different sparsity S = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9] been applied to study how each sparsity affects the model accuracy, size, inference speed and power consumption, followed by fine-tuning to recover the model accuracy.
- TensorFlow post-training Quantization to carry out post training quantization onto the trained base model and pruned model to reduce the weights of the model from float32 to float 16, integer8 and float fallback. While float32, float16 and integer8 are standard variable types, float fallback is a special format of the TensorFlow library. It automatically quantize most operations to integer type but any operations that lack a suitable integer implementation are automatically left in float32 (fallback).

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.

C. Implementation and Evaluation Details

The ResNet-50 model was trained for 80 epochs on an NVIDIA A5000 GPU using 416×416 images, a batch size of 16, and sigmoid focal cross entropy ($\gamma = 2$). All models were subsequently quantized and converted into TensorFlow Lite (TFLite) formats (F32, F16, INT8, Float Fallback) for embedded inference.

A set of 30 images (defective and non-defective) was tested. Power consumption was derived from raw oscilloscope data (trigger, current, voltage, time) by filtering out non-inference states ($trigger < 2V$) and calculating instantaneous power (I×V). This power was summed across all 30 images. Inference time was determined by measuring and averaging the interval between the rising and falling edges of the trigger signal.

IV. Results and Discussion

This section presents the experimental findings and performance evaluation of the baseline model against its pruned, quantized, and combined pruned-quantized variants. The evaluation considers classification accuracy, power consumption, inference time, and model size. Analysis highlights the trade-offs between computational efficiency and predictive performance resulting from pruning and quantization.

A. Comparison of standalone Quantization and Pruning

Table III presents the comparison between the base models and the tested quantization schemes. The FP32 and FP16 schemes maintained the lowest accuracy loss relative to INT8 and Fallback. Specifically, FP32 retained the original Keras model size, whereas FP16 halved the size while preserving original inference time and power consumption. INT8 demonstrated the best efficiency trade-offs: it achieved a four-fold model size reduction and the shortest inference time (equivalent to Fallback), with a modest, acceptable accuracy drop compared to FP16 and FP32. This reduction in inference time is paramount for industrial AOI, as higher throughput is required to match the rapid pace of semiconductor manufacturing lines without creating production bottlenecks. Consequently, INT8 is suitable for resource-constrained deployment, while FP16 is recommended when maximizing accuracy is the primary production requirement.

TABLE III
Performance of ResNet-50 under different quantization schemes

Precision	Performance Metrics			
	Top-1 Acc. (%)	*Power (W/img)*	*Inference Time (s/img)*	*Size (MB)*
Base Keras	99.42	15.91	3.06	90.12
FP32	91.57	15.91	3.06	90.12
FP16	92.03	15.54	3.06	48.85
INT8	94.11	7.68	1.79	25.13
Fallback	94.80	7.54	1.79	25.23

The effect of pruning sparsity on power, inference time, and model size is presented in Table IV. Increased sparsity leads to marginal changes in per-image latency and power but a monotonic decrease in model size. Since the inference time remains constant, pruning does not significantly improve inspection throughput, meaning it is more useful for devices that have limited storage rather than those requiring faster processing speeds. With an accuracy drop of only approx. 5% at 0.9 sparsity, pruning is demonstrated to be an effective strategy for achieving significant memory savings for embedded deployment at moderate levels. Conversely, while maximizing memory reduction, very high sparsity results in a noticeable accuracy penalty.

TABLE IV
Performance of ResNet-50 under different pruning sparsity levels

Sparsity	Performance Metrics			
	Top-1 Acc. (%)	*Power (W/img)*	*Inference Time (s/img)*	*Size (MB)*
Base Keras	99.42	15.91	3.06	90.12
0.1	99.31	15.61	3.06	84.71
0.2	98.88	15.22	3.06	80.61
0.3	98.46	14.87	3.06	77.34
0.4	98.01	14.47	3.06	70.54
0.5	97.46	13.90	3.06	54.38
0.6	96.42	13.32	3.06	45.08
0.7	95.15	13.75	3.06	37.02
0.8	95.15	13.22	3.06	28.07
0.9	93.76	13.19	3.07	19.14

B. Comparison of combined techniques

Having established the individual effects of pruning (reducing parameters and storage via sparsity) and quantization (compressing precision for reduced memory footprint and faster inference), we combined these techniques for further model optimization. We applied pruning at different sparsity levels with post-training quantization at various precisions to identify models achieving an optimal trade-off. As a result, 40 models with varying levels of pruning and quantization have been created to conduct an extensive comparison. The analysis utilizes three quadrant charts (Figure 2 – Figure 4), constructed using thresholds for accuracy, inference time, model size, and power consumption. The quadrants categorize models as: QI (high accuracy, low resource), QII (high accuracy, high resource), QIII (low accuracy, low resource), and QIV (low accuracy, high resource).

Visualized in Figure 2, the analysis of accuracy versus power consumption reveals distinct trends for different model configurations. Most of the pruned FP32 and FP16 models exhibit high power consumption, falling predominantly within quadrants QIV and QII. However, as sparsity increases, the power consumption decreases, as exemplified by the models with floating point 16 quantization and sparsity of 0.8 (labeled FP16_0.8) and models with floating point 16 quantization and sparsity of 0.2 (labeled FP16_0.2). Conversely, models quantized to INT8 and FF maintain low power consumption. Further compression of these models, however, shows limited additional power savings due to the Raspberry Pi's system floor-level power consumption and the brief duration

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.

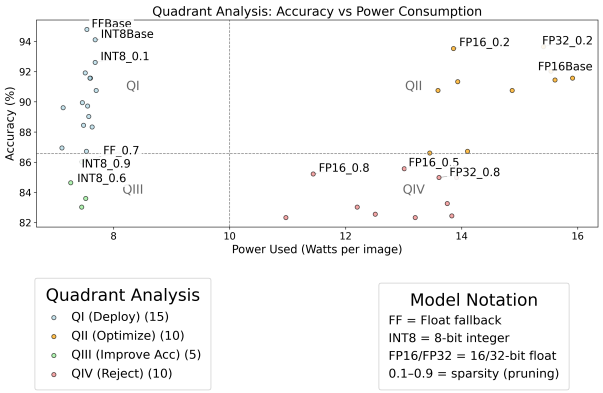


Fig. 2. Quadrant analysis: accuracy against power consumption

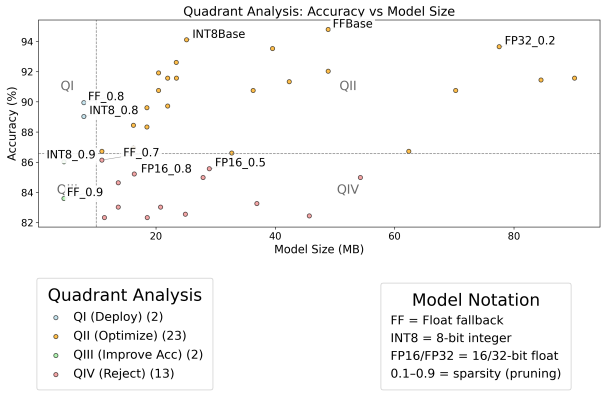


Fig. 4. Quadrant analysis: accuracy against model size

of the inference bursts. Therefore, the optimized models in QI, namely FFBase, INT8Base, and INT8_0.1, are the most suitable candidates for deployment as they offer an optimal balance of high accuracy and low power consumption.

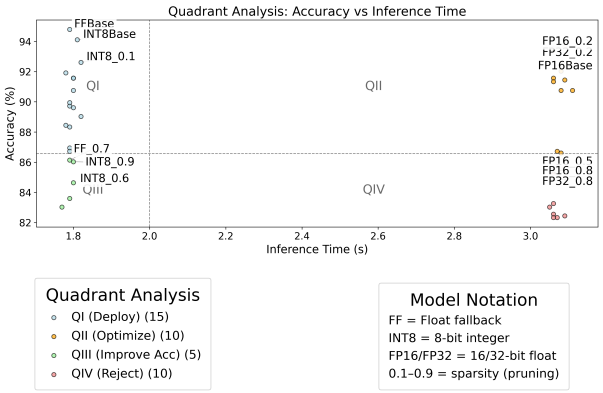


Fig. 3. Quadrant analysis: accuracy against inference time

Figure 3 demonstrates that INT8 and FF models, resulting from pruning and quantization, achieve the shortest inference time with a minimal loss in accuracy when compared to FP32 and FP16 models, which require an inference time of approximately 3.2 seconds per test image. Models in QI are therefore promising candidates for real-time inspection deployment where latency is critical. Models in QII require further optimization using techniques like structured pruning or operator fusion to reduce inference time, as pruning alone has shown a minimal impact on accuracy. This further compression could significantly reduce latency. Finally, models in QIII and QIV, comprising heavily pruned FP32, INT8, FP16, and FF models, are unsuitable for real-time production due to either excessively long inference time or substantial accuracy degradation during compression.

As pictured in Figure 4, quantization and pruning significantly impact model size, with INT8 and FF models achieving up to a 4× reduction relative to the FP32 base model while maintaining competitive accuracy. Further model compression is achieved by highly sparse models, such as FF_0.8 and INT8_0.8, which reach a size of 10MB. Consequently, models in QI are ideal for deployment due to their lightweight nature and high accuracy. Models in QII also achieve high accuracy but remain larger, making them suitable for environments with fewer memory constraints. Conversely, models in QIII and QIV are unsuitable for deployment due to the combination of substantial accuracy degradation and inflated size.

The combined analyses identify several models suitable for embedded deployment, specifically those that are severely pruned and quantized into INT8 and FF precision. These models consistently reside in QI across all metrics, offering low model size, power consumption, and inference time with only a marginal impact on accuracy. Conversely, models in QIII and QIV highlight the trade-off boundaries of these compression techniques, indicating either over-aggressive pruning leading to a significant accuracy drop or compression without commensurate gains in efficiency metrics. This finding confirms that while pruning and quantization have individual impacts on model metrics, their combined application is crucial for identifying the most optimal configurations that successfully balance accuracy and efficiency for the constraints of the real-time semiconductor task.

V. Conclusion and Future Work

This research proposes a cascaded classification and detection framework for chip defects, employing a combination of weight-magnitude-based pruning and post-training quantization on a pre-trained ResNet-50 model. Experimental evaluations, validated through power and inference time measurements, confirm the effectiveness of each technique for deployment on the Raspberry Pi 4B. The results demonstrate outstanding real-time performance regarding low model size, power consumption, and inference time. From a sustainability perspective, minimizing power consumption is crucial for reducing the overall carbon footprint of high-volume manufacturing facilities and promoting greener production lines in the semiconductor industry.

For future work, we plan to investigate a wider array of compression techniques, including structured pruning, knowledge distillation, mixed-precision quantization, and compiler-level

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.

optimizations. To rigorously analyze transferability, we will extend the hardware evaluation beyond the Raspberry Pi 4B to industrial-grade platforms such as Industrial PCs, Jetson, Coral, and FPGA/ASIC. This expanded analysis will focus on creating a standardized energy efficiency benchmark across all tested platforms, allowing the semiconductor industry to directly quantify and reduce the carbon impact of deploying deep learning models to AOI systems. Furthermore, the study will be applied to the second stage of the cascaded process, incorporating the YoloX model to allow for a broader comparison of model efficiency and accuracy trade-offs.

VI. Acknowledgments

The research has partly been supported by the Dieter Schwarz Stiftung, Heilbronn Germany. Further support was received through the project Integrated International Degree Programmes with Double Degrees, funded by the German Academic Exchange Service (DAAD) and additional funding was provided through the Erasmus+ KA171-HED project, supported by the European Commission. Microsoft's M365 CoPilot based on GPT-4o and Gemini-3 Pro has been used for grammar and language editing.

References

[1] R. Huang, D. Zhan, X. Yang, B. Zhou, L. Tang, N. Cai, H. Wang, and B. J. M. Qiu, "Atnet: A defect detection framework for x-ray images of dip chip lead bonding," vol. 14, no. 7, p. 1375, 2023.

[2] K. Imoto, T. Nakai, T. Ike, K. Haruki, and Y. J. I. T. o. S. M. Sato, "A cnn-based transfer learning method for defect classification in semiconductor manufacturing," vol. 32, no. 4, pp. 455–459, 2019.

[3] S. Cheon, H. Lee, C. O. Kim, and S. H. Lee, "Convolutional neural network for wafer surface defect classification and the detection of unknown defect class," *IEEE Transactions on Semiconductor Manufacturing*, vol. 32, no. 2, pp. 163–170, 2019.

[4] Y. D. Wong, A. Gu, and X. Shi, "Advanced automatic defect detection of semiconductor chips: Overcoming complexity and diversity with addnet."

[5] V. P. Dhruv, K. B. Ravi, and A. O. Assad, "Deep learning-based detection, classification, and localization of defects in semiconductor processes," *Journal of Micro/Nanolithography, MEMS, and MOEMS*, vol. 19, no. 2, p. 024801, 2020. [Online]. Available: https://doi.org/10.1117/1.JMM.19.2.024801

[6] C. J. Jeong, M. S. Han, J. W. Kim, H. H. Hong, H. J. Lim, D. Lee, S. J. Hong, G. J. J. o. t. S. Kim, and D. Technology, "Design of automatic defect classification system for wafer edge defect inspection," vol. 23, no. 4, pp. 183–189, 2024.

[7] B. M. Haddad, S. F. Dodge, L. J. Karam, N. S. Patel, and M. W. Braun, "Locally adaptive statistical background modeling with deep learning-based false positive rejection for defect detection in semiconductor units," *IEEE Transactions on Semiconductor Manufacturing*, vol. 33, no. 3, pp. 357–372, 2020.

[8] C.-Y. Huang, I.-C. Lin, and Y.-L. Liu, "Applying deep learning to construct a defect detection system for ceramic substrates," vol. 12, no. 5, p. 2269, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/5/2269

[9] T. Schlosser, M. Friedrich, F. Beuth, and D. Kowerko, "Improving automated visual fault inspection for semiconductor manufacturing using a hybrid multistage system of deep neural networks," *Journal of Intelligent Manufacturing*, vol. 33, no. 4, pp. 1099–1123, 2022. [Online]. Available: https://doi.org/10.1007/s10845-021-01906-9

[10] C. Phua and L. B. Theng, "Semiconductor wafer surface: Automatic defect classification with deep cnn," in *2020 IEEE REGION 10 CONFERENCE (TENCON)*, Conference Proceedings, pp. 714–719.

[11] J. L. Gómez-Sirvent, F. L. d. l. Rosa, R. Sánchez-Reolid, A. Fernández-Caballero, and R. Morales, "Optimal feature selection for defect classification in semiconductor wafers," *IEEE Transactions on Semiconductor Manufacturing*, vol. 35, no. 2, pp. 324–331, 2022.

[12] S. Nag, D. Makwana, S. C. T. R, S. Mittal, and C. K. Mohan, "Wafersegclassnet - a light-weight network for classification and segmentation of semiconductor wafer defects," *Computers in Industry*, vol. 142, p. 103720, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0166361522001178

[13] S. Wang, S. Yan, Q. Shen, C. Luo, L. Li, J. Ai, S. Ding, Q. Xia, Z. Li, and Q. Chen, "A real-time automated defect classification system based on two-step convolutional neural network," in *Journal of Physics: Conference Series*, vol. 2337. IOP Publishing, Conference Proceedings, p. 012006.

[14] X. Zheng, T. Sang, R. Chen, and Z. Zeng, "Enhancing semiconductor chip image defect classification using deep learning and segmented defect region information," in *2023 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM)*, Conference Proceedings, pp. 204–209.

[15] Q. Zhang, K. Zhang, K. Pan, and W. Huang, "Image defect classification of surface mount technology welding based on the improved resnet model," *Journal of Engineering Research*, vol. 12, no. 2, pp. 154–162, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2307187724000348

[16] D. Na, J. Gu, J. Park, Y. Song, J. Moon, S. Ha, S. J. J. o. t. S. Hong, and D. Technology, "Deep learning-based defect detection in cu-cu bonding processes," vol. 23, no. 2, pp. 135–142, 2024.

[17] Y. Kim, J. S. Lee, and J. H. Lee, "Automatic defect classification using semi-supervised learning with defect localization," *IEEE Transactions on Semiconductor Manufacturing*, vol. 36, no. 3, pp. 476–485, 2023.

[18] X. Zhu, S. Wang, J. Su, F. Liu, and L. Zeng, "High-speed and accurate cascade detection method for chip surface defects," *IEEE Transactions on Instrumentation and Measurement*, vol. 73, pp. 1–12, 2024.

[19] Y. Chen, B. Zheng, Z. Zhang, Q. Wang, C. Shen, and Q. Zhang, "Deep learning on mobile and embedded devices: State-of-the-art, challenges, and future directions," vol. 53, no. 4

[20] S. Ameen, K. Siriwardana, and T. J. a. p. a. Theodoridis, "Optimizing deep learning models for raspberry pi," 2023.

[21] Y. Lecun, J. Denker, and S. Solla, *Optimal Brain Damage*, 1989, vol. 2.

[22] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural network," in *Neural Information Processing Systems*, Conference Proceedings.

[23] Y. Guo, A. Yao, and Y. J. A. i. n. i. p. s. Chen, "Dynamic network surgery for efficient dnns," vol. 29, 2016.

[24] M. Zhu and S. J. a. p. a. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," 2017.

[25] S. Anwar, K. Hwang, and W. J. A. J. o. E. T. i. C. S. Sung, "Structured pruning of deep convolutional neural networks," vol. 13, no. 3, pp. 1–18, 2017.

[26] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. J. a. p. a. Kautz, "Pruning convolutional neural networks for resource efficient inference," 2016.

[27] J.-H. Luo, J. Wu, and W. Lin, "Thinet: A filter level pruning method for deep neural network compression," in *Proceedings of the IEEE international conference on computer vision*, Conference Proceedings, pp. 5058–5066.

[28] J.-H. Luo and J. J. P. R. Wu, "Autopruner: An end-to-end trainable filter pruning method for efficient deep model inference," vol. 107, p. 107461, 2020.

[29] M. Zhu and S. J. a. p. a. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," 2017.

[30] M. Courbariaux, Y. Bengio, and J.-P. J. a. p. a. David, "Training deep neural networks with low precision multiplications," 2014.

[31] P. Kluska and M. Zieba, "Post-training quantization methods for deep learning models," ser. Intelligent Information and Database Systems. Springer International Publishing, Conference Proceedings, pp. 467–479.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 27,2026 at 10:27:07 UTC from IEEE Xplore. Restrictions apply.